

Task Description

Develop a "Library Management System" to demonstrate Object-Oriented Programming concepts.

Part A: Classes and Objects

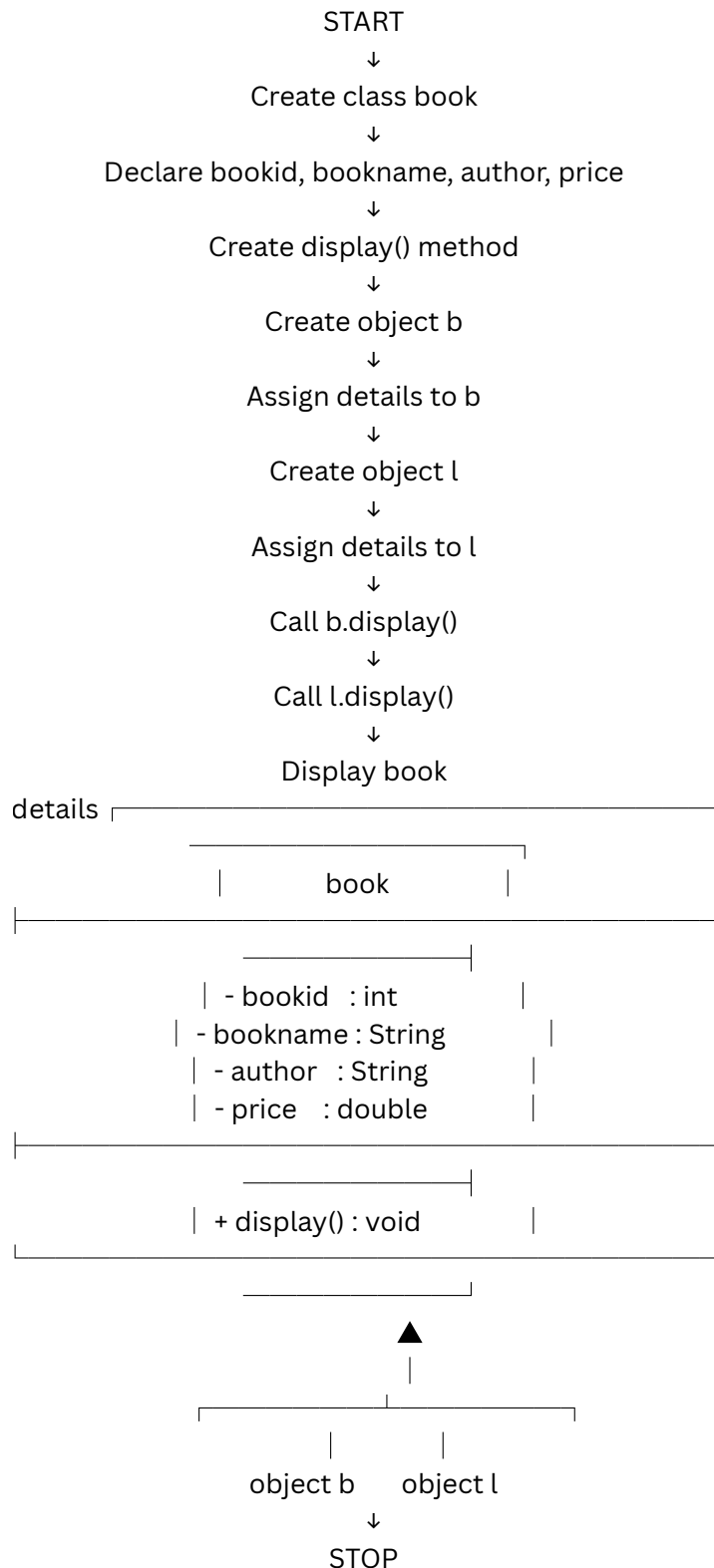
Create a class named Book containing:

- Book ID
- Book Name
 - Author
 - Price

Create objects and display their details.

Steps

- Create a class named book.
- Declare four data members:
 - bookid
 - bookname
 - author
 - price
- Create a method display() to print book details.
- Create another class named Task2.
- Create the first object b of the book class.
- Assign values to the first book.
- Create the second object l.
- Assign values to the second book.
- Call display() using b.
- Call display() using l.
- Display both book details.



```

        class book {
            int bookid;
            String bookname;
            String author;
            double price;
            void display(){

                System.out.println("book id name" +bookid);
                System.out.println("bookname is" + bookname);
                System.out.println("book authoris " + author);
                System.out.println("the price of book is" + price);
            }
        }

        public class Task2 {
            public static void main(String[] args) {
                book b = new book();
                b.bookid = 244;
                b.bookname = "java learns";
                b.author = "james gosling";
                b.price = 250.00;
                book l = new book();
                l.bookid = 28;
                l.bookname = "learn";
                l.author = " suresh";
                l.price = 45.50;

                b.display();
                l.display();
            }
        }

```

```

        book id name244
        bookname isjava learns
        book authoris james gosling
        the price of book is250.0

```

```

        book id name28
        bookname islearn
        book authoris suresh
        the price of book is45.5

```

Part B: Constructors and Encapsulation

- Create both default and parameterized constructors.
 - Declare data members as private.
- Access them using Getter and Setter methods.

Steps

Create a class book.

Declare bookid, bookname, author, and price as private.

Create a parameterized constructor.

Use this keyword to initialize the data members.

Create setter methods to modify the private data.

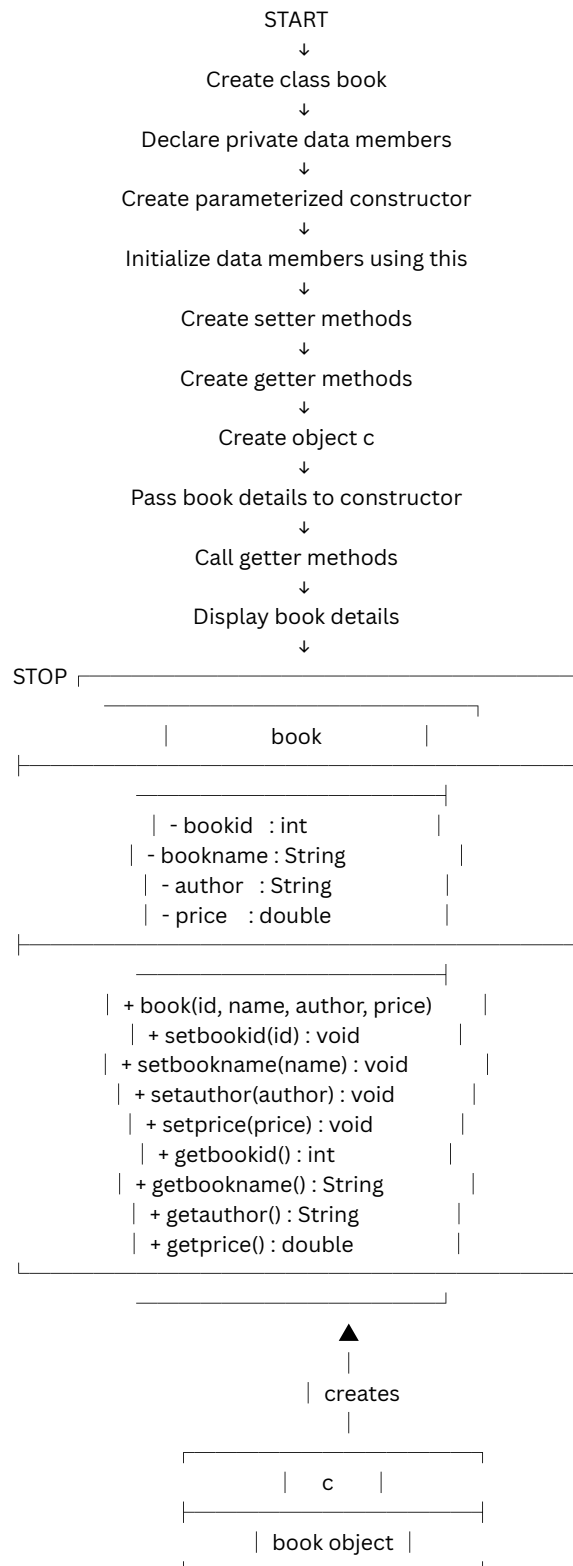
Create getter methods to access the private data.

Create an object c using the constructor.

Use getter methods to retrieve the book details.

Display the details.

Stop.



```

class book{
    private int bookid;
    private String bookname;
    private String author;
    private double price;
public book( int id , String name , String author , double
    price){
    this.bookid = id;
    this.bookname = name;
    this.author = author;
    this.price = price;
    }
    void setbookid(int id ){
        bookid = id;
    }
    void setbookname(String name){
        bookname = name;
    }
    void setauthor(String author ){
        this.author = author;
    }
    void setprice(double price){
        this.price = price;
    }
    int getbookid(){
        return bookid;
    }
    String getbookname(){
        return bookname;
    }
    String getauthor(){
        return author;
    }
    double getprice(){
        return price;
    }
    }

    public class Task2 {
        public static void main(String[] args) {
            book c = new book(244, "java", "james", 350.50);
            System.out.println("book id is"+ c.getauthor());
            System.out.println("book name is"+c.getbookname());
            System.out.println("author is "+ c.getauthor());
            System.out.println("price is "+c.getprice());

        }

    }

```

Book ID is 244
 Book name is java
 Author is james
 Price is 350.5

Part C: Inheritance

Create the following classes:

- Person
- Student (inherits Person)
- Faculty (inherits Person)

Display the inherited properties using objects.

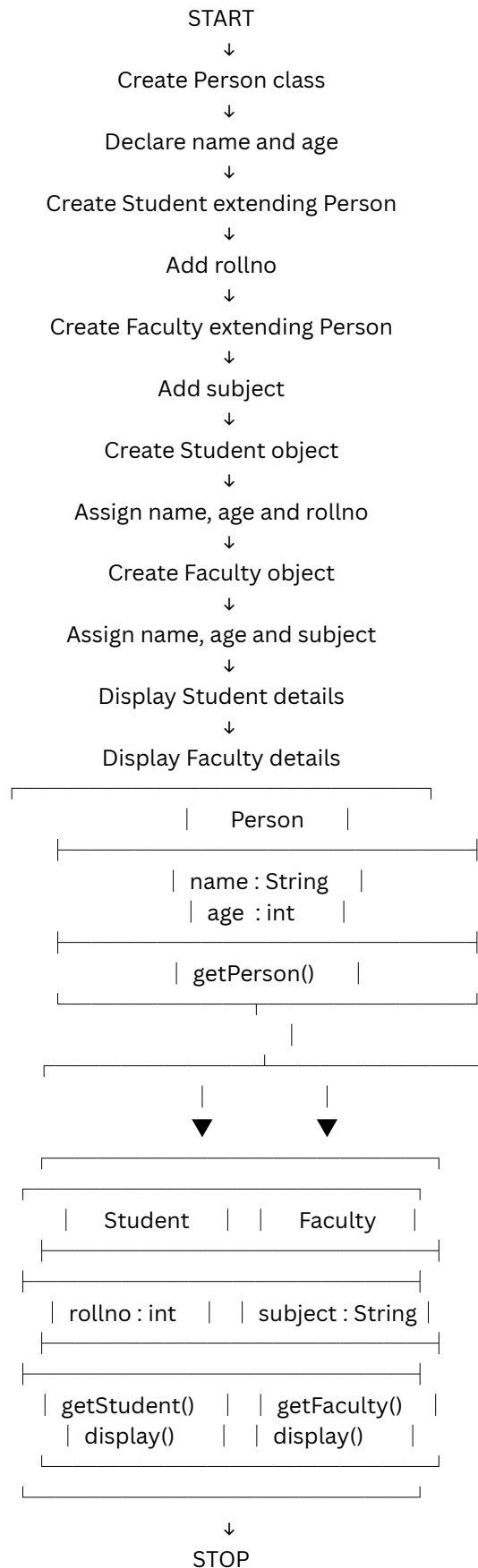
Part D: Polymorphism

Implement:

- Method Overloading for calculating the area of different shapes.
- Method Overriding by creating a Vehicle class and overriding the display() method in Car and Bike classes.

Steps

Create a Person class.
Declare common properties name and age.
Create a Student class using extends Person.
Add rollno to Student.
Create a Faculty class using extends Person.
Add subject to Faculty.
Create objects of Student and Faculty.
Assign values to inherited properties.
Assign values to their own properties.
Display all properties.




```

class person{
    String name;
    int age;
void getperson(String name , int age){
    this.name = name;
    this.age = age;
    }
}

class student extends person{
    int rollno;
    void getstudent(int rollno){
        this.rollno = rollno;
    }
    void show(){
System.out.println("---student details---");
        System.out.println("name is " + name);
        System.out.println("age is"+ age);
        System.out.println("rollno is" + rollno);

    }
}

class faculty extends person{
    String subject;
    void getfaculty(String subject){
        this.subject = subject;
    }

    void show(){
        System.out.println("--- faculty details---");
System.out.println("name of faculty is"+ name);
        System.out.println(" age of " +age);
        System.out.println("subject is " + subject);
    }
}

public class Task21 {
public static void main(String[] args) {
    student s= new student();
    s.getperson("hemanth", 19);
    s.getstudent(244);
    faculty f= new faculty();
    f.getperson("srinu", 35);
    f.getfaculty("programming");
    s.show();
    f.show();

}
}

```

--- Student Details ---

Name: Hemanth

Age: 19

Roll No: 244

--- Faculty Details ---

Name: Srinu

Age: 35

Subject: Programming

Create a class Area.

Create an area() method with one double parameter for a circle.

Create another area() method with two double parameters for a rectangle.

Create another area() method with three double parameters for a triangle.

Call the appropriate overloaded method.

Display the calculated areas.

START



Create Area class



Define area(radius)



Define area(length, breadth)



Define area(base, height, dummy)



Call overloaded methods

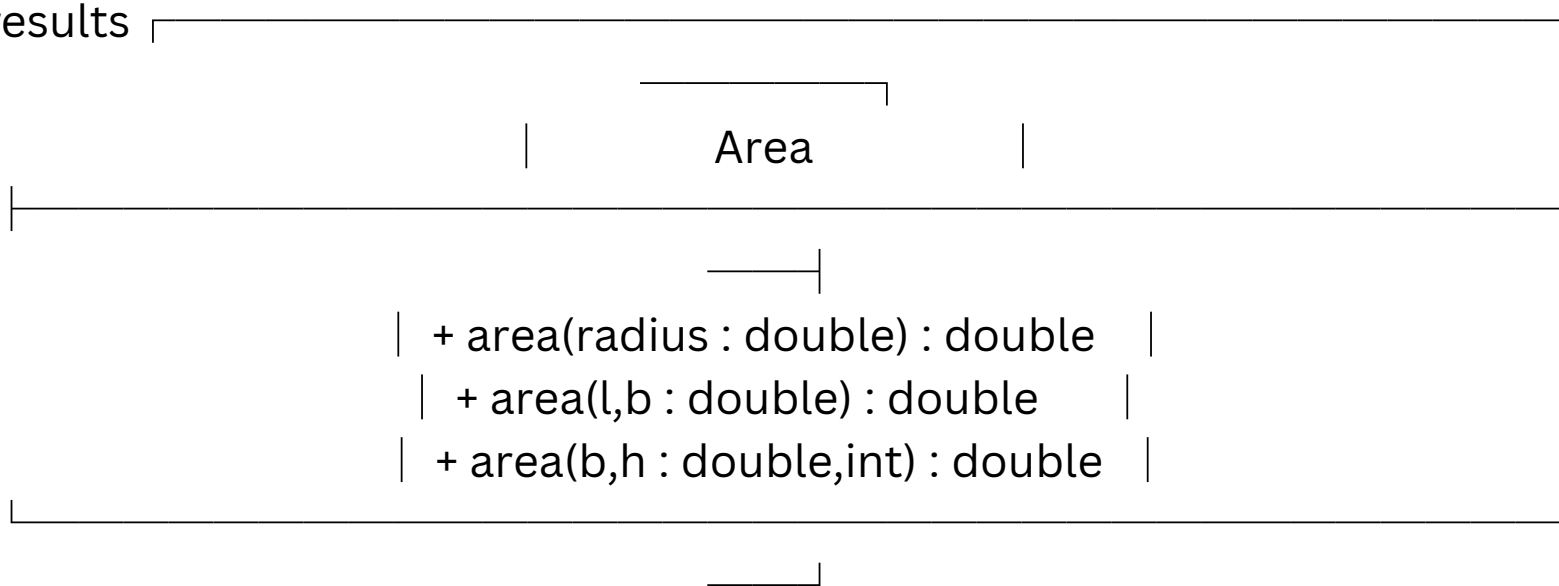


Calculate areas



Display

results



STOP

```

class Area {

    // Area of Circle
    double area(double radius) {
        return 3.14 * radius * radius;
    }

    // Area of Rectangle
    double area(double length, double breadth) {
        return length * breadth;
    }

    // Area of Triangle
    double area(double base, double height, int triangle) {
        return 0.5 * base * height;
    }
}

```

```

public class PartD1 {
    public static void main(String[] args) {

        Area a = new Area();

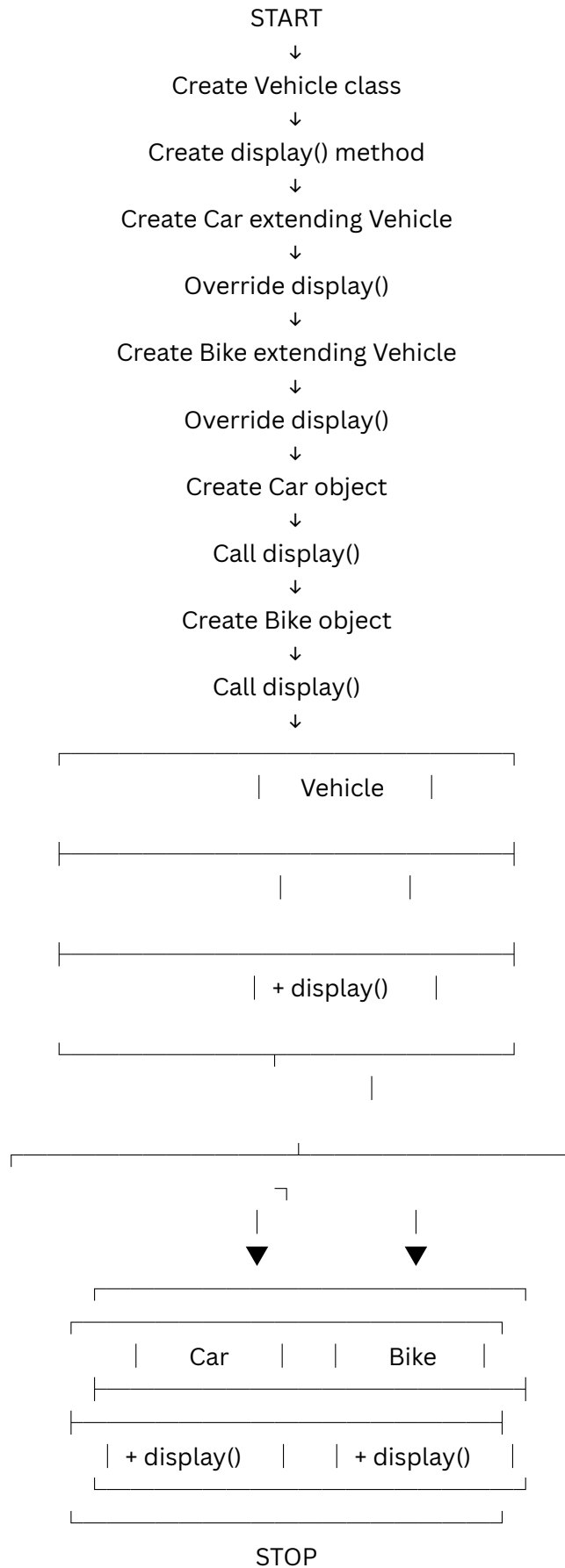
        System.out.println("Area of Circle = " + a.area(5.0));
        System.out.println("Area of Rectangle = " +
            a.area(10.0, 5.0));
        System.out.println("Area of Triangle = " +
            a.area(10.0, 6.0, 1));
    }
}

```

Area of Circle = 78.5
 Area of Rectangle = 50.0
 Area of Triangle = 30.0

Steps

- Create a parent class Vehicle.
- Define a display() method.
- Create Car extending Vehicle.
- Override the display() method in Car.
- Create Bike extending Vehicle.
- Override the display() method in Bike.
- Create objects of Car and Bike.
- Call display() for both objects.
- Display their respective details.



```

class Vehicle {

    void display() {
System.out.println("This is a vehicle");
    }
}

class Car extends Vehicle {

    @Override
    void display() {
        System.out.println("This is a Car");
System.out.println("Car has four wheels");
    }
}

class Bike extends Vehicle {

    @Override
    void display() {
        System.out.println("This is a Bike");
System.out.println("Bike has two wheels");
    }
}

public class PartD2 {
public static void main(String[] args) {

```

```

    Vehicle v1 = new Car();
    Vehicle v2 = new Bike();

```

```

        v1.display();
        v2.display();
    }
}

```

```

    This is a Car
    Car has four wheels
    This is a Bike
    Bike has two wheels

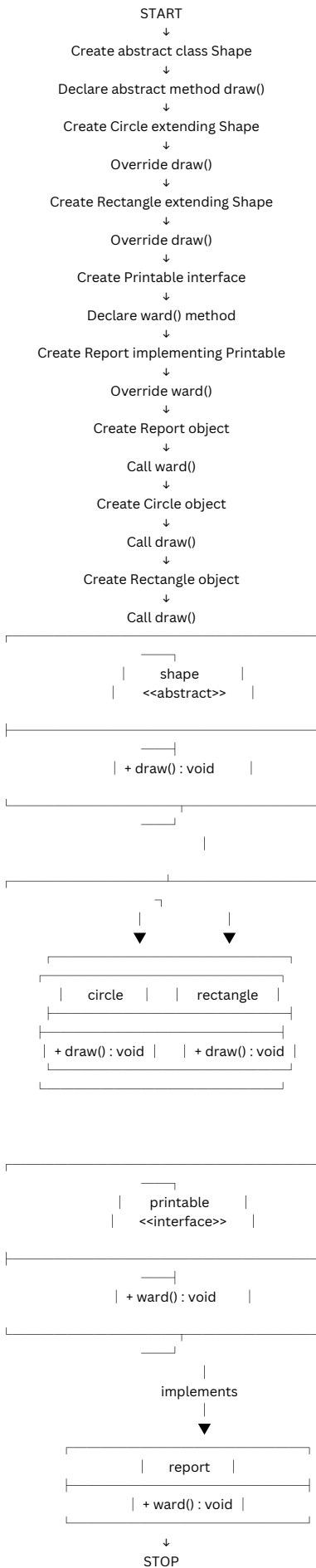
```

Part E: Abstraction and Interfaces

- Create an abstract class Shape containing an abstract method draw().
- Implement the draw() method in Circle and Rectangle classes.
- Create an interface Printable and implement it in a class named Report.

2. Steps

- Create an abstract class shape.
- Declare an abstract method draw().
- Create the circle class extending shape.
- Override the draw() method in circle.
- Create the rectangle class extending shape.
- Override the draw() method in rectangle.
- Create an interface printable.
- Declare the ward() method inside the interface.
- Create the report class implementing printable.
- Override ward() in the report class.
- Create objects of report, circle, and rectangle.
- Call their respective methods.
- Display the output.



```

abstract class shape{
    abstract void draw();
}

```

```

class circle extends shape{
    @Override
    public void draw(){
System.out.println("the drawing is circle");
    }
}

```

```

class rectangle extends shape{
    @Override
    public void draw(){
System.out.println("th edrawing is rectangle");
    }
}

```

```

interface printable{
    void ward();
}
class report implements printable{
    @Override
    public void ward(){
System.out.println("the interface is implenents");
    }
}

```

```

public class Task23 {
public static void main(String[] args) {
    report r = new report();
        r.ward();
    circle a = new circle() ;
        a.draw();
    rectangle t = new rectangle();
        t.draw();
    }
}

```

The interface is implemented
The drawing is a circle
The drawing is a rectangle